

AI 활용 기술 문서

LATENT — 레벨을 미리 만들지 않는 퍼즐

저장소 github.com/TaraGamesDev/latent-nan2026 참가 개인

1. 한 문장

캐주얼 퍼즐의 원가는 레벨뱅크이고, 그 원가에서 비싼 쪽은 만드는 일이 아니라 만든 레벨이 풀리는지 검증하는 일입니다. 레벨을 미리 만들지 않으면 그 검증 단계가 통째로 사라집니다.

LATENT의 타일은 덮여 있는 동안 무늬가 없습니다. 관측되지 않았으니 아직 존재할 이유가 없습니다. 위가 걷혀 드러나는 순간, AI가 플레이어의 그 시점 상황을 읽고 무늬를 씁니다 — 그때 “이 레벨은 끝까지 클리어 가능한가”를 부등식 하나로 보장하면서, 사후 검증이 필요 없습니다. 애초에 불가능한 상태를 만들지 않기 때문입니다.

2. 왜 이 방향인가 — 비싼 것은 생성이 아니라 검증

소코반 레벨 자동 생성은 논문 주제입니다. MCTS로 생성하고, 난이도는 “문제 분해 지표”나 “사람의 탐색을 흉내낸 계산 모델”로 추정합니다. 이유는 단순합니다 — 소코반은 PSPACE-complete라 만든 레벨이 풀리는지, 얼마나 어려운지를 싸게 알 방법이 없습니다. 그래서 사람이 검수해야 하고, 그게 원가입니다.

그러므로 장르 선택의 기준은 “재미있을까”가 아니라 “난이도와 풀림을 근사가 아니라 정확히, 싸게 알 수 있는가”가 됩니다. 이 축으로 후보 셋을 직접 구현해 재고 골랐습니다.

2.1 후보 비교 (측정값)

후보	검증 처리량	정확성
A · STAMP GF(2) 토글 퍼즐	85,228 레벨/초	완전 정확. 가우스 소거 한 번으로 풀림 여부·최소 수·해의 개수($2^{\text{영공간차원}}$)가 전부 나옵니다. 유일해 레벨은 풀리는 것의 37%.
B · MAGNET 당겨 붙이는 퍼즐	26 레벨/초	불완전. 레벨당 탐색 노드 28,988개. 깊이 6에서 6.7%만 풀리고, 나머지는 “안 풀린다”가 아니라 <i>모른다</i> 입니다.
C · 타일 + 자연 결정 (채택)	470,000 결정/초	검증 단계가 없음. 배정 시점의 부등식으로 클리어 가능성이 구성상 보장됩니다.

B가 A보다 3,300배 느립니다. 이것이 이 프로젝트가 하려는 이야기의 축소판입니다. 소스는 `src/probe/`에 그대로 있고 `npx tsx src/probe/a-stamp.ts` 등으로 재현됩니다.

A는 “난이도를 정확히 계산한다”는 주장이 가장 강했지만 C를 골랐습니다. 이유는 검증 비용을 0으로 만드는 쪽이 더 근본적이기 때문이고, 시장에서 검증된 장르 위에서 그렇게 할 수 있기 때문입니다.

3. 아키텍처

타일이 드러날 때마다

- [스킬 추정기] `src/ai/skill.ts`
`regret = value(솔버의 최선수) - value(플레이어가 고른 수)`
→ θ (숙련도), {계획성 · 정돈 · 템포}, {좌절도 · 지루함}
- [배정기 · 의도] `src/ai/assigner.ts`
 $\theta + \text{감정} + \text{레벨 진행 램프} \rightarrow \text{목표 난이도}$
트레이 위험도가 높으면 목표와 무관하게 구제
후보: 즉시완성 / 짝만들기 / 보드와짝 / 새무늬
- [배정기 · 시공] 같은 파일
★ 미결정 타일 수 ≥ 3 의 배수를 맞추는 데 필요한 타일 수
이 부등식을 깨는 후보는 기각됩니다
- [글래스박스 패널] `src/ui/panel.ts`
위 전부를 화면에 그대로 노출

모듈	역할
<code>core/tiles.ts</code>	더미 기하, 덮임 관계, 트레이 규칙
<code>core/rng.ts</code>	시드 기반 PRNG — 시드만 있으면 런 전체가 재현됩니다
<code>ai/solver.ts</code>	국면 평가와 수순 순위. 보드를 복제하지 않고 해석적으로 계산합니다
<code>ai/skill.ts</code>	수순당 후회 → 숙련도 추정, 감정 상태 분리
<code>ai/assigner.ts</code>	무늬 결정 + 완결성 검사
<code>ai/policy.ts</code>	정책 상수. 빌드에 구워짐
<code>ai/bots.ts</code>	봇 페르소나 — 콜드스타트 문제의 해법
<code>probe/</code>	2.1의 후보 비교를 재현하는 실측 코드

4. 핵심 설계 세 가지

4.1 수순당 후회(regret)로 실력을 잴다

“레벨을 클리어했는가”, “몇 분 걸렸는가”는 몇 분에 한 번 도착하고 그 레벨이 마침 쉬웠는지에 오염됩니다. 대신 **모든 탭을, 그 탭이 이루어진 바로 그 국면에서** 채점합니다.

$$\text{regret} = (\text{최선수의 가치} - \text{플레이어가 고른 수의 가치})$$

그 국면에서의 선택지 가치 편차

국면이 통제되어 있으므로 “쉬워서 잘한 것”과 “실제로 잘 둔 것”이 섞이지 않습니다. 선택지가 전부 동등한 국면은 **정보량 0**으로 처리해 추정을 흔들지 않고, 표본이 쌓일 때까지는 $\text{samples} / (\text{samples} + 10)$ 비율로 사전확률에서 벗어납니다 — 첫 판부터 최대 압박이 걸리지 않도록.

4.2 의도와 시공을 분리한다

배정기는 “지금 이 순간이 어떻게 느껴져야 하는가”만 정합니다. 실제로 무늬를 쓰기 전에 완결성 부등식이 별도로 검사합니다. 플레이어 평가를 하는 쪽은 틀려도 되지만, 타일을 쓰는 쪽은 틀리면 안 되기 때문입니다. 추정이 어긋나면 난이도가 조금 안 맞을 뿐이지만, 배정이 잘못되면 그 레벨은 영원히 클리어할 수 없습니다.

4.3 약속은 작게, 대신 반드시 지킨다

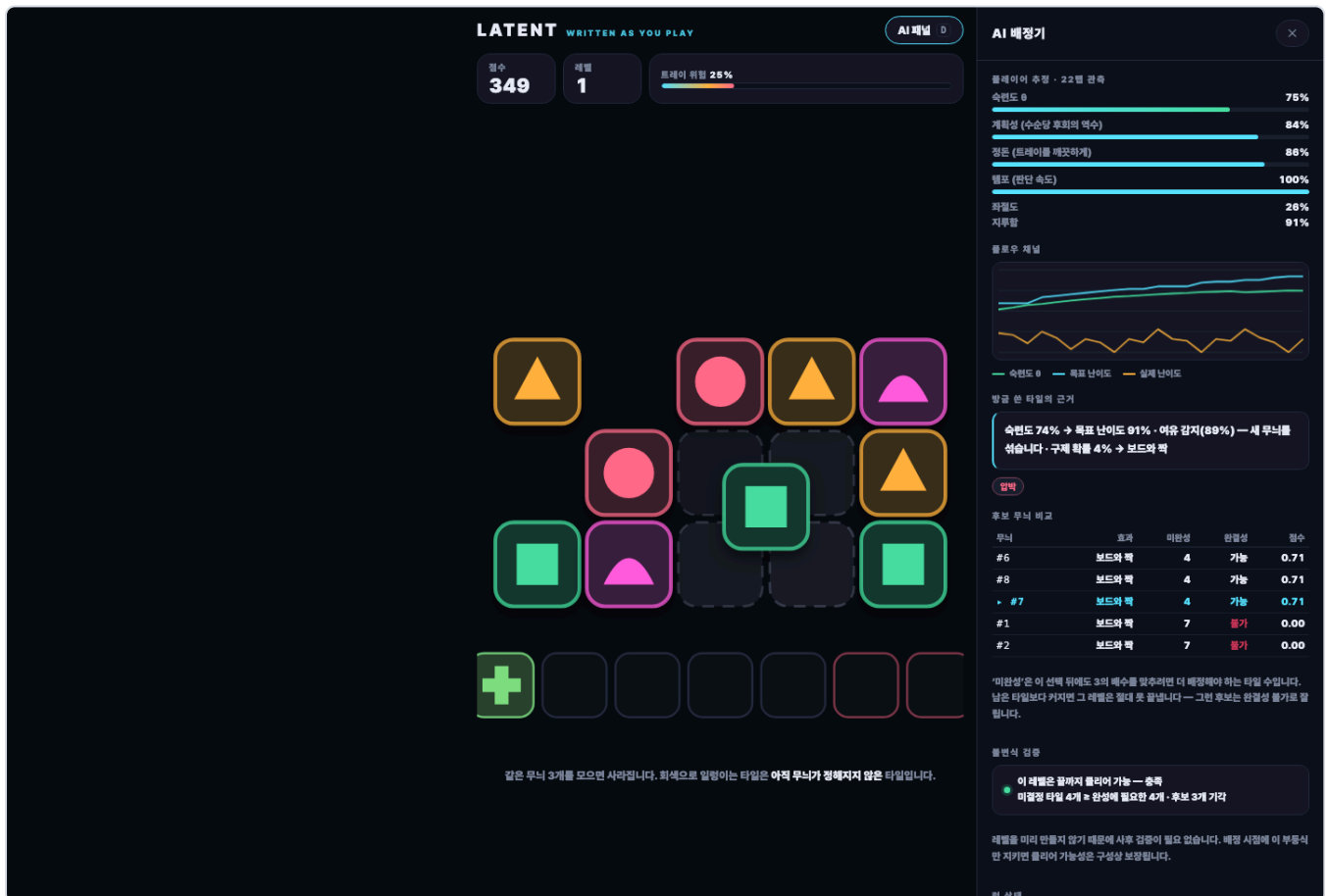
불변식 — 이 레벨은 끝까지 클리어 가능하다.

모든 무늬는 최종적으로 3의 배수가 되어야 합니다. 그러지 못하면 남은 타일은 영원히 사라지지 않습니다. 그래서 매 배정마다 미결정 타일 수 $\geq$ 열려 있는 무늬를 닫는 데 필요한 타일 수를 검사하고, 이를 깨는 후보는 기각합니다.

다만 질 수는 있습니다. 그래야 맞습니다. 계산해 보면 절대 지지 않게 만드는 것은 불가능합니다 — 보이는 무늬가 k 종이면 최악의 경우 각 2장씩 $2k$ 장까지 쌓이므로 k 가 4만 되어도 7칸이 찹니다. k 를 3으로 묶으면 게임이 무의미해집니다. 그래서 지는 것은 플레이어의 선택이지 배정의 실패가 아닙니다. 이 구분을 문서와 패널 양쪽에서 일부러 지켰습니다.

5. 클래스박스 — AI가 자기 판단을 보여준다

조용히 결정하는 시스템은 자기 작업을 보여줄 수 있는 시스템보다 가치가 낮습니다. 특히 이 장르는 플레이어들이 “게임이 나를 조작한다”고 의심하는 것으로 유명합니다. 그렇다면 숨길 것이 아니라 열어 두는 편이 낫습니다.



실제 플레이 중 캡처. 좌: 색이 있는 타일은 무늬가 결정된 것, 점선은 아직 무늬가 없는 타일. 우: 22턴 관측 시점의 추정(0 75%, 계획성 84%, 정돈 86%), 플로우 채널, 방금 쓴 타일의 근거, 그리고 후보 무늬 비교표 — 여기서 #1과 #2가 완결성 ‘불가’로 기각된 것이 보입니다. 불변식이 실제로 후보를 자르고 있다는 증거입니다. 맨 아래에 “미결정 타일 4개 $\geq$ 완성에 필요한 4개 · 후보 3개 기각”.

6. AI 도구 사용 내역

단계	사용 도구	내용
런타임 (배포 빌드)	없음	LLM 호출 없음. 게임 안의 “AI”는 전부 결정론적 코드 — 솔버, 추정기, 배정기. 네트워크·API 카·계정 불필요.
설계·구현	Claude Code (<code>claude-opus-5</code>)	코딩 협업 도구. 규칙 설계 토론, 엔진·AI·UI 구현, 헤드리스 계측 도구 작성, 설계 결함 진단. 6.2 참조.
에셋 생성	없음	이미지·사운드·폰트를 생성하지도, 사용하지도 않았습니 다. 모든 그래픽은 Canvas 2D로 코드가 직접 그립니다.

6.1 런타임에 LLM을 쓰지 않은 이유

- 심사자가 링크만 눌러도 돌아가야 합니다. 사전과제 요건이 “별도 유료 라이선스 없이 실행 가능”입니다. 런타임 LLM은 키가 없으면 게임 자체가 죽습니다.
- 결정이 초당 수십 회입니다.** 타일 하나가 드러날 때마다 판단하므로 네트워크 왕복이 성립하지 않습니다. 측정된 배정 지연은 탭당 평균 0.015ms입니다.
- 타일을 쓰는 코드는 틀리면 안 됩니다.** 4.2의 분리 원칙에 따라, LLM은 *정책을 설계하는 자리*에 두고 *타일에 무늬를 쓰는 자리*에는 두지 않았습니

6.2 개발 단계의 AI 활용과 실제 지시

게임 규칙, 장르 선택, 스코프 결정은 사람이 내렸고, AI는 구현·계측·진단을 담당했습니다. 작업 방식에서 결정적이었던 지시는 두 가지입니다.

“UI를 만들기 전에 헤드리스로 엔진을 검증한다.”

보고만 하는 벤치가 아니라 **단언**하는 벤치를 만들게 했습니다. 이 한 가지가 여러 설계 결함을 화면에 닿기 전에 잡아냈습니

“재미있는지는 짐작하지 말고 계측한다.”

`src/probe/` 의 후보 비교와, 이전 규칙을 폐기하게 만든 의사결정 깊이 진단이 여기서 나왔습니

전체 진행은 저장소 커밋 기록에 남아 있습니다. 규칙을 여러 차례 갈아엎는 동안 **AI 층**(추정기·글래스박스·벤치·페르소나)은 한 번도 재작성되지 않았습니 — 이 프로젝트의 자산이 어디에 있는지 보여주는 사실이라고 생각합니다.

7. 검증

`npm run bench` 는 봇 페르소나로 전체 게임을 실제 **tap()** 경로로 돌립니다. 심사자가 플레이하는 바로 그 코드에서 나온 수치입니다. 50런 × 3페르소나:

페르소나	클리어 레벨	점수	탭	완성	외톨이	θ	탭당 ms
novice	1.44	1,656	89	27.4	1.68	0.559	0.032
casual	6.26	10,643	494	162.2	1.16	0.629	0.014
expert	26.06	66,052	2,573	856.7	0.54	0.909	0.015

- OK — 완결성 불변식이 모든 런의 모든 탭에서 유지됨
- OK — novice와 casual은 전부 패배로 끝남 (트레이가 실제로 물어야 함)
- OK — 실력이 보상됨: 1.44 → 26.06 레벨 (18배)
- OK — θ가 페르소나를 마땅한 순서로 정렬: 0.559 < 0.629 < 0.909

7.1 계측이 화면에 닿기 전에 잡아낸 것들

증상	원인과 수정
초기 시안에서 턴당 선택지 2.5개, 58% 동점	판이 차면 대부분의 말이 영구히 썬기박히는 이동 규칙 → 규칙 전면 교체. 짐작이 아니라 의사결정 깊이 진단으로 판정.
유능한 봇이 4,000탭까지 안 죽음	종료 강제력 부재 → 강도 램프 도입. 이후 밀도가 자기평형을 이루는 구조적 문제까지 드러나 규칙 자체를 재검토.
지연 결정 1차 시안에서 서투른 플레이어가 100% 패배	배정기가 자기 장부만 보고 실제 트레이를 안 봄 → 트레이를 직접 읽도록 수정. “플레이어 상황에 따라”가 말이 아니라 코드가 됨.
난이도 다이얼이 절벽 (0.35→100%, 0.7→0%)	구제 등급이 3단계뿐이라 목표값이 경계를 넘으면 팀 → 확률적 선택 + 위험 구간 강제 게이트로 매끄러운 기울기 확보.
숙련자가 레벨마다 물려진 난이도로 재시작	난이도 상한이 레벨 진행에만 연동됨 → 상한 자체를 θ에 연동.

8. 외부 에셋 및 오픈소스 출처

외부 에셋 없음. 이미지·사운드·폰트·아이콘 파일이 저장소에 하나도 없습니다. 타일의 여덟 가지 무늬(원·삼각형·사각형·마름모·별·육각형·십자·아치)는 전부 Canvas 2D 경로로 코드가 그립니다. 서체는 사용자 기기의 시스템 폰트를 사용합니다.

패키지	라이선스	범위	용도
<code>vite</code> ^6.0.5	MIT	devDependency	개발 서버 및 정적 빌드
<code>typescript</code> ^5.7.2	Apache-2.0	devDependency	타입 검사
<code>tsx</code> ^4.19.2	MIT	devDependency	벤치·프로브 실행
<code>@types/node</code>	MIT	devDependency	타입 정의
<code>@anthropic-ai/sdk</code>	MIT	devDependency	오프라인 도구용. 배포 번들에 미포함

package.json 의 **dependencies** 는 비어 있습니다. 위 다섯은 전부 개발 전용입니다. 빌드 산출물은 JS 25KB · CSS 8KB(gzip 11KB · 2KB)이고 외부 요청이 0건입니다.

9. 한계와 다음 단계

- **사람 데이터가 없습니다.** 정책은 봇 페르소나에 맞춰져 있습니다. 봇은 사람의 대용이지 사람이 아닙니다. 다음 단계는 실제 플레이 로그로 페르소나를 재적합하는 것입니다.
- **추정이 세션 내로 한정됩니다.** 계정·서버가 없으므로 θ 는 매 런 사전확률에서 다시 시작합니다. 세션 간 프로필을 유지하면 첫 30초의 정확도가 크게 올라갑니다. 해커톤 스코프에서 의도적으로 제외했습니다.
- **더미 기하는 아직 절차적이지 않습니다.** 층별 계단형 배치를 쓰고 있습니다. 기하까지 생성 대상으로 넣으면 “레벨 디자인 자동화”의 남은 절반이 채워집니다.
- **정책 상수는 손으로 정했습니다.** 봇 페르소나를 목적함수에 걸어 탐색하는 오프라인 튜너를 이전 규칙에서 구현해 30% 개선을 얻은 적이 있고, 이 규칙으로 이식하는 것이 가장 가까운 다음 작업입니다.